High Performance Computing – MapReduce Assignment

Orazgali Azamat, BDA-2402

The programs were run with Python 3.11.9 on Windows. Python3 is not available in Windows, so I use the standard python command.

Why .bat files are used: the task commands are written for a Linux/WSL terminal (cat, wc -l, line continuation with \). In the Windows Command Prompt these commands do not exist, so for every part there is a .bat file that runs the same pipeline with the Windows equivalents). This is also more convenient than typing the long pipelines or queries every time.

Part 1: Understanding Map and Reduce

1. Mapper Output

Document	Mapper Output
doc1	(data,1), (science,1), (data,1)
doc2	(science,1), (needs,1), (data,1)

2. Shuffle and Sort

data → [1, 1, 1]
science → [1, 1]
needs → [1]

3. Reduce

(data, 3)
(science, 2)
(needs, 1)

1. What is the input key to the mapper in a document-processing MapReduce application?
The identifier of the input record, for example the document name/ID or the position (offset) of the line in the file.

2. What is the input value?
The content of the record, i.e. the text of the document or of the line.

3. What key is produced by the Word Count mapper?
The word itself (the value is 1 in this case).

4. What does the shuffle and sort stage do?
It collects the intermediate pairs from all mappers, sorts them by key and groups all values with the same key together, so that each key arrives at a reducer with the list of all its values.

5. Why must all intermediate values having the same key reach the same reducer?
Otherwise the result for that key would be split between several reducers and each would output only a partial count. Only when one reducer sees all values of a key can it produce the correct final result.

Part 2: Implementing Word Count

Files: input.txt (5 lines, the example text from the task), mapper.py, reducer.py. The mapper output is saved in map_output.txt (32 pairs) and the final result in output.txt.

1. Does sort calculate the word counts?
No. It only reorders the lines. The counting is done by the reducer.

2. What useful property does sorting provide to the reducer?
All pairs with the same word become adjacent. The reducer can read the lines one by one, add up the counts while the word stays the same, and output the total when the word changes. It does not have to keep all words in memory.

Final Word Count output (output.txt):

Word	Count
across	1
and	1
be	1
can	3
computing	1
data	3
datasets	1
designed	1
distributed	2
faster	1
for	1
is	1
large	1
machines	1
many	1
map	1
mapreduce	2
parallel	2
process	2

processes	1
processing	1
reduce	1
systems	1
using	1

Part 3: Follow the Data Flow

Input (dataflow.txt):
red blue red
blue green
red green blue

Stage	Data
Map Output	(red,1) (blue,1) (red,1) (blue,1) (green,1) (red,1) (green,1) (blue,1)
After Shuffle and Sort	blue → [1, 1, 1] green → [1, 1] red → [1, 1, 1]
Final Reduce Output	(blue,3) (green,2) (red,3)

1. Does a mapper need to know the final number of occurrences of a word?
No. It only outputs 1 for every word it sees.

2. Does one mapper need the output of another mapper before it can process its own input?
No. Mappers work on their own input split independently.

3. Which stage brings together values generated by different mappers?
The shuffle and sort stage.

4. Is the reducer executed before all relevant mapper outputs become available?
No. The reduce function can only start after all mappers have finished, because any mapper could still produce a value for the key. (The copying of map outputs may start earlier, but the reduce step waits.)

Part 4: Combiner

Files: part1.txt ("data data science data"), part2.txt ("science data science") and combiner.py (same aggregation logic as the reducer).

combined1.txt:
data 3
science 1

combined2.txt:
data 1
science 2

Final result after cat combined1.txt combined2.txt | sort | reducer:
data 4
science 3

This is the same as running the reducer without a combiner (data appears 4 times and science 3 times).

File	Records without combiner	Records with combiner
part1.txt	4	2
part2.txt	3	2
Total	7	4

The command wc -l map_output.txt from the task gives 32 lines, but this file is the mapper output of input.txt from Part 2, so the fair comparison is the mapper output of the same part file (4 and 3 lines) against combined1.txt and combined2.txt (2 lines each).

1. Does a combiner change the final Word Count result?
No. The final result is the same (data 4, science 3).

2. Does it reduce the number of intermediate records that may need to be transferred?
Yes. Here the records went from 7 to 4. For large texts with many repeated words the reduction is much bigger.

3. Why can reducing intermediate data reduce network traffic?
Intermediate data is sent over the network from the mappers to the reducers during the shuffle. Fewer records mean fewer bytes to send, so the shuffle is faster.

4. Why can a summation operation safely use a combiner?
Addition is associative and commutative, so the numbers can be added in any grouping and order. Adding partial sums locally and then adding those partial sums gives the same total.

Part 5: Partitioning

partition(k) = (sum of the ASCII codes of the characters of k) mod 2

Key	ASCII sum	Reducer
data	410	R0
science	730	R0
parallel	845	R1
computer	879	R1
hadoop	635	R1
reduce	632	R0

1. Can the same intermediate key be sent to two different reducers in a normal per-key reduction?
No. The partition function always gives the same result for the same key, so all its values go to one reducer.

2. Can two different keys be sent to the same reducer?
Yes. For example data, science and reduce all go to R0.

3. Why is partitioning necessary when several reducers are used?
Each mapper must decide independently to which reducer every key goes. The partitioner gives a fixed rule, so all values of one key end up at the same reducer and the keys are spread over all reducers.

4. Does partitioning guarantee that all reducers receive exactly the same amount of work?
No. It only decides where keys go. Some keys can have many more values than others (for example a very frequent word), and the hash can put more keys on one reducer, so the load can be unbalanced.

Part 6: Inverted Index

Files: documents.txt, index_mapper.py (emits word and document ID) and index_reducer.py (collects the document IDs of each word without duplicates). Final output (index_output.txt):

Word	Documents
computing	doc1,doc4
data	doc2,doc3,doc4
is	doc3
large	doc2
many	doc1
mapreduce	doc2
parallel	doc1,doc3
process	doc4
processes	doc2
processing	doc3

processors	doc1
systems	doc4
useful	doc3
uses	doc1

1. What is the intermediate key in this application?
The word.

2. What is the intermediate value?
The document ID where the word appears.

3. Why must all occurrences of one word be sent to the same reducer?
The reducer has to build the complete list of documents for that word. If the occurrences were split between reducers, each would output only part of the list, and the same word would appear in several output lines.

4. How is this application different from Word Count?
In Word Count the value is the number 1 and the reducer adds the values. In the inverted index the value is a document ID and the reducer builds a list of unique document IDs instead of a number.

Part 7: MapReduce Runtime

Operation	Programmer / Runtime
Define the mapper	Programmer
Define the reducer	Programmer
Schedule map tasks	Runtime
Group intermediate values by key	Runtime
Shuffle intermediate data	Runtime
Detect failed workers	Runtime
Restart failed tasks	Runtime
Choose what the application computes	Programmer

1. Why is hiding scheduling and communication details useful to the programmer?
The programmer only writes the map and reduce functions and does not have to deal with distributing tasks, moving data between machines or handling failures. The same simple code can then run on many machines.

2. What happens between the map and reduce phases?
The shuffle and sort: the map outputs are partitioned, sent to the right reducers, sorted and grouped by key.

3. Why can intermediate data movement become expensive?
The map outputs can be very large and have to be written to disk and sent over the network from every mapper to every reducer. The network is much slower than local memory.

Part 8: Data Locality and Distributed Storage

1. Why does a MapReduce scheduler try to execute a map task close to the physical location of its input data?
Reading data from the local disk is much faster than sending it over the network, and it keeps the network free for other traffic.

2. What is meant by "moving computation to the data"?
Instead of copying large data to the machine with the program, the small program is sent to the machines that already store the data and runs there.

3. What is the purpose of a distributed file system in a MapReduce environment?
It stores very large files split into blocks across many machines, so that the data can be read in parallel by many tasks and survives machine failures.

4. What is the role of the HDFS NameNode?
It is the master that stores the metadata: the directory tree, the list of blocks of every file and on which DataNodes each block is stored.

5. What is the role of an HDFS DataNode?
It stores the actual data blocks on its local disks, serves read and write requests for them, and regularly reports to the NameNode.

6. Does the NameNode normally transfer the actual file contents between the client and DataNodes?
No. The NameNode only tells the client which DataNodes have the blocks. The client reads and writes the data directly from/to the DataNodes.

7. Why are file blocks replicated on several machines?
So that the data is still available if a machine or disk fails, and so that more tasks can read the same block from different machines (better data locality).

8. How can replication improve fault tolerance?
If a DataNode fails, the block can be read from another replica, and the system creates a new copy to restore the number of replicas. A failed map task can be restarted on another machine that has a copy of its input.

Part 9: Short Discussion

1. What is the difference between the Map phase and the Reduce phase?
The Map phase processes each input record independently and turns it into intermediate key-value pairs. The Reduce phase receives all values for one key and combines them into the final result. Map works on pieces of the input, Reduce works on groups of values with the same key.

2. What is the purpose of the shuffle and sort stage?
It moves the intermediate pairs from the mappers to the reducers and groups them by key. Because of this every reducer gets each of its keys together with all of its values, sorted by key.

3. Why are key-value pairs central to the MapReduce model?
The key tells the system how to group and distribute the data: all values with the same key go to the same reducer. Both the input and the output of map and reduce are key-value pairs, so the runtime can handle any application in the same way.

4. What is the purpose of a combiner?
A combiner does local aggregation of the map output on the mapper side before the shuffle. It reduces the amount of intermediate data that has to be sent over the network, without changing the final result.

5. What is the difference between a combiner and a partitioner?
A combiner reduces the data by aggregating values of the same key locally on one mapper. A partitioner does not change the data; it only decides which reducer receives each key.

6. Why is Word Count naturally parallelizable?
Each line or document can be split into words independently, so the map tasks need no communication. Counting is a sum, which can be done in parts and combined at the end in any order.

7. Give one example of a problem that is well suited to MapReduce.
Counting how many times each URL appears in very large web server log files. Every log line can be processed independently and the counts per URL are just summed.

8. Give one example of a problem that would be difficult to express efficiently as a single MapReduce job.
Iterative algorithms such as PageRank or k-means clustering. They need many passes over the data, and each pass would be a separate MapReduce job that writes its result to disk and reads it again, which is slow.